

INSTITUTO FEDERAL
Fluminense
Campus Campos Centro

aula.tex

SISTEMAS DIGITAIS

Notas de aula

```
$ make aulas
> compiling slides ..... ok
> checking alunos ..... ok
```

[Prof. Dr. Vinicius Cavalcanti] // [IFFluminense] // 14 de novembro de 2026

[v 0.4]

EMENTA

```
> ls aulas/
00  apresentação da disciplina
01  sistemas de numeração
02  circuitos lógicos
03  circuitos combinacionais
04  operações e circuitos aritméticos
05  unidade lógica e aritmética
06  circuitos sequenciais
07  multiplexadores e demultiplexadores
08  memória
```

> cd seção_00

00

APRESENTAÇÃO DA DISCIPLINA

> ls bibliografia/

01 TOCCI, R.J.; WIDMER, N.S.; MOSS, G.L. Sistemas digitais.
02 IDOETA, I.V.; CAPUANO, F.G. Elementos de eletrônica digital.
03 MALVINO, A.B.; LEACH, D.P. Eletrônica digital.
04 TAUB, H. Circuitos digitais e microprocessadores.

> ls simulações/

> logisim

CONTEXTUALIZAÇÃO

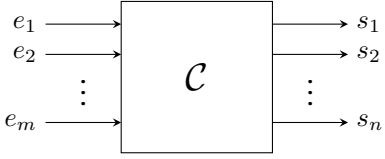
Os sistemas digitais representam a base fundamental de toda a informática moderna. Na prática, tudo o que manipulamos em um computador é traduzido e processado na forma de sinais digitais.

> Aplicações:

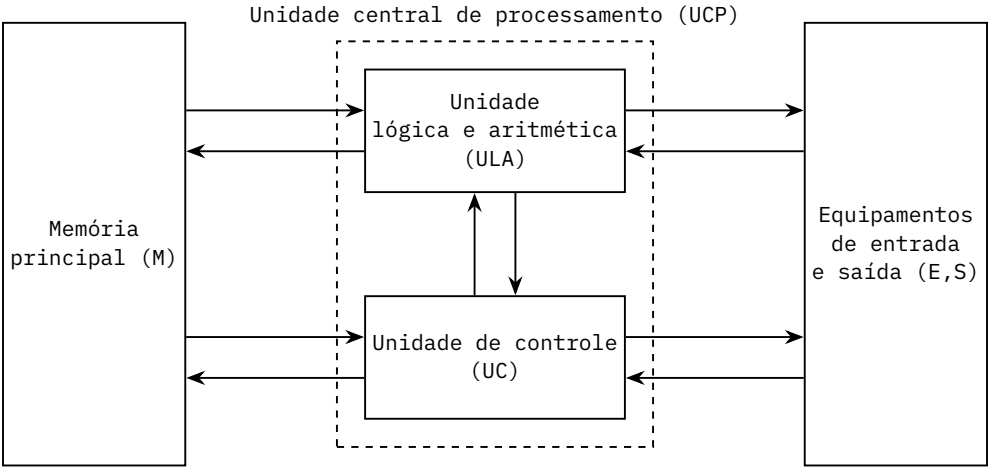
- Computação;
- Automação industrial e robótica;
- Telecomunicações;
- Sistemas embarcados ...

Sistema

Um sistema é uma entidade física ou um modelo matemático que relaciona o fluxo de sinais de entrada e saída.

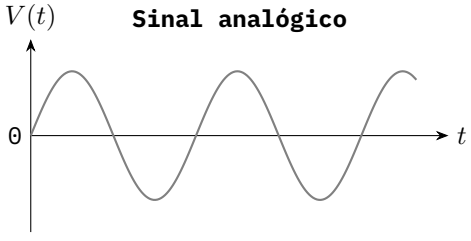


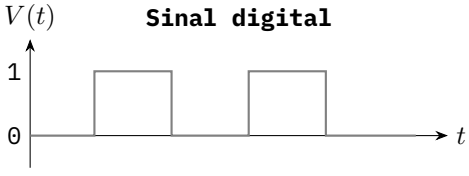
ARQUITETURA GENÉRICA DE UM COMPUTADOR



SINAIS ANALÓGICOS E DIGITAIS

Sinais de diferentes naturezas exigem modelos matemáticos e estruturas de hardware completamente distintas para serem processados de forma correta e eficiente.





Um computador digital é uma máquina eletrônica programável que processa dados e executa operações lógicas e aritméticas a partir de instruções armazenadas. Toda informação é representada por sinais discretos.

> Confiabilidade e simplicidade em relação a um sistema decimal na distinção de estados;
> A arquitetura é baseada em transistores que funcionam como chaves eletrônicas.

DESVANTAGENS

O mundo real é analógico, e a digitalização sempre introduz algum erro.
O processamento de sinais digitalizados introduz alguma latência.

> run --conversão

> Converter a variável física em um sinal elétrico (analógico)
> Converter o sinal elétrico (analógico) para a forma digital
> Processar (manipular) a informação digital
> Converter as saídas digitais de volta para a forma analógica

> cd seção_01

01

SISTEMAS DE NUMERAÇÃO

UM SISTEMA DE NUMERAÇÃO

Um sistema de numeração é um sistema de escrita para expressar números sem o uso de palavras; isto é, uma notação matemática para representar números de um determinado conjunto, utilizando dígitos (sistema posicional) ou outros símbolos (sistema aditivo) de maneira consistente.

> Sistema aditivo
  - Números romanos

Número romano	Valor
I	1
V	5
X	10
L	50
C	100
D	500
M	1.000

UM SISTEMA POSICIONAL QUALQUER

Cada símbolo se encaixa em uma posição específica, representando uma potência de uma base fixa.

O valor do número é a soma de cada algarismo que o compõe, considerando a posição em que ele se encontra. Um número inteiro x com n dígitos, num sistema de base b, pode ser representado da seguinte forma

$$x = (d_{n-1}d_{n-2}\cdots d_1 d_0)_b = \sum_{k=0}^{n-1} d_k b^k$$

EXEMPLO

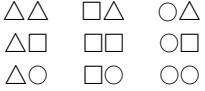
> run --exemplo

> $b = 3$

> $d = \{\triangle, \square, \bigcirc\}$

> $n = 2$

> Contagem



SISTEMA DECIMAL

O sistema decimal é composto por 10 símbolos $d = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$; com os quais é possível expressar qualquer número.

> run --exemplo

> $x = (437)_{10}$

> $x = 4 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0$

> $n = 3$

0	10	100
1	11	101
2	12	102
3	⋮	⋮
4	19	198
5	20	199
6	21	200
7	⋮	⋮
8	98	998
9	99	999

SISTEMA BINÁRIO

Infelizmente, o sistema decimal não é prático para sistemas digitais, pois projetar equipamentos eletrônicos que diferenciem 10 níveis de tensão é muito complexo. Por outro lado, é muito mais simples e eficiente construir circuitos baseados em apenas dois níveis de tensão.

O sistema binário é composto por 2 símbolos $d = \{0, 1\}$; com os quais é possível expressar qualquer número.

CONTAGEM BINÁRIA

> run --exemplo

> $x = (1011)_2$

> $x = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

> $x = 8 + 0 + 2 + 1$

> $x = (11)_{10}$

> $n = 4$

0000	0100	1000	1100
0001	0101	1001	1101
0010	0110	1010	1110
0011	0111	1011	1111

SISTEMA HEXADECIMAL

O sistema hexadecimal é composto por 16 símbolos $d = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$; com os quais é possível expressar qualquer número.

Esse sistema é utilizado, principalmente, como uma representação "abreviada" de números binários.

> run --exemplo

> $x = (2AF)_{16}$

> $x = 2 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0$

> $x = 512 + 160 + 15$

> $x = (687)_{10}$

CONVERSÃO DE BINÁRIO PARA DECIMAL

> Método

2^{n-1}	2^{n-2}	$\dots$	2^1	2^0
d_{n-1}	d_{n-2}	$\dots$	d_1	d_0

$$d_{n-1}2^{n-1} + d_{n-2}2^{n-2} + \cdots + d_1 2^1 + d_0 2^0$$

> Exemplo

- $x = (11011)_2$

2^4	2^3	2^2	2^1	2^0
1	1	0	1	1

$$x = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 16 + 8 + 2 + 1 = (27)_{10}$$

CONVERSÃO DE HEXADECIMAL PARA DECIMAL

> Método

16^{n-1}	16^{n-2}	$\cdots$	16^1	16^0
d_{n-1}	d_{n-2}	$\ldots$	d_1	d_0

$$d_{n-1}16^{n-1} + d_{n-2}16^{n-2} + \cdots + d_1 16^1 + d_0 16^0$$

> Exemplo

- $x = (110)_{16}$

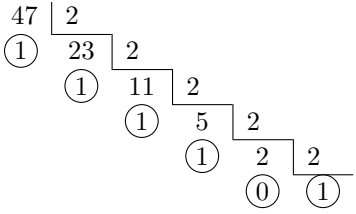
16^2	16^1	16^0
1	1	0

$$x = 1 \cdot 16^2 + 1 \cdot 16^1 + 0 \cdot 16^0 = 256 + 16 = (272)_{10}$$

CONVERSÃO DE DECIMAL PARA BINÁRIO

> Método
- A conversão de números inteiros decimais utiliza divisões sucessivas por 2 até que se obtenha um quociente igual a 0.
- Registra-se o resto de cada divisão.
- O resultado em binário é obtido escrevendo o último resto como o bit mais significativo. Os outros dígitos seguem na ordem reversa até chegar-se ao 1º resto.

> Exemplo
- $x = (47)_{10}$



$$x = (101111)_2$$

CONVERSÃO DE DECIMAL PARA HEXADECIMAL

> Método
  - A conversão de números inteiros decimais utiliza divisões sucessivas por 16 até que se obtenha um quociente igual a 0.

> Exemplo
  - $x = (1000)_{10}$

$$\begin{array}{r|l} 1000 & 16 \\ \cline{2-2} (8) & 62 \quad \begin{array}{|l} 16 \\ \hline (14) \quad (3) \end{array} \end{array}$$

$$x = (3E8)_{16}$$

CONVERSÃO DE HEXADECIMAL PARA BINÁRIO

> Método
  - Cada dígito é convertido para seu equivalente binário de quatro dígitos.

> Exemplo
  - $x = (9F2)_{16}$

9	F	2
1001	1111	0010

$$x = (100111110010)_2$$

CONVERSÃO DE BINÁRIO PARA HEXADECIMAL

> Método
  - O número binário é dividido em grupos de quatro bits, e cada grupo é convertido no seu dígito hexadecimal equivalente.

> Exemplo
  - $x = (1110100110)_2$

0011	1010	0110
3	A	6

$$x = (3A6)_{16}$$

CÓDIGO BCD

O código BCD (binary-coded-decimal) descreve a codificação de números decimais na qual cada dígito decimal é representado por um número fixo de bits, geralmente quatro.

> Exemplos

- $x = (874)_{10}$

8	7	4
1000	0111	0100

$$x = (100001110100)_{\text{BCD}}$$

- $x = (100110111)_{\text{BCD}}$

0001	0011	0111
1	3	7

$$x = (137)_{10}$$

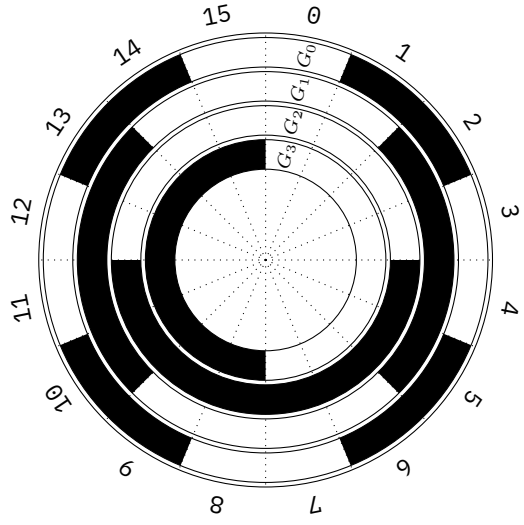
CÓDIGO DE GRAY

O código Gray é um sistema binário não ponderado onde apenas um bit muda entre dois números consecutivos.

O código de Gray é amplamente utilizado para evitar interpretar erroneamente uma entrada variável.

B_2	B_1	B_0	G_2	G_1	G_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

ENCODER ROTATIVO



UNIDADES DE PROCESSAMENTO DE DADOS

- Nibble: 4 bits
- Byte: 8 bits
- Word: é a unidade natural de informação usada em uma arquitetura particular.
  - 8 bits
  - 16 bits
  - 32 bits
  - 64 bits

Um bit de paridade é um bit extra que é anexado a um conjunto de dados para detectar erros na transmissão ou armazenamento. Funciona como uma verificação de integridade do dado.

> **Paridade par:** O bit de paridade é ajustado para que o número total de bits 1 (incluindo o próprio bit de paridade) seja par.
  - Se a mensagem tiver um número ímpar de 1s, o bit de paridade será 1.
  - Se a mensagem já tiver um número par de 1s, o bit de paridade será 0.

> **Paridade ímpar:** O bit de paridade é ajustado para que o número total de bits 1 (incluindo o próprio bit de paridade) seja ímpar.
  - Se a mensagem tiver um número par de 1s, o bit de paridade será 1.
  - Se a mensagem já tiver um número ímpar de 1s, o bit de paridade será 0.

LISTA DE PROBLEMAS

```
> ls problemas/
> 2.1 (b), (d), (f)
> 2.2 (b), (d), (f)
> 2.3 (b)
> 2.4 (b), (d), (f)
> 2.5 (b), (d), (f)
> 2.6 (b), (d), (f)
> 2.7 (b), (d), (f)
> 2.15
> 2.19 (b), (d), (f)
> 2.21 (b), (d), (f)
> 2.28
```

> cd seção_02

02
CIRCUITOS LÓGICOS

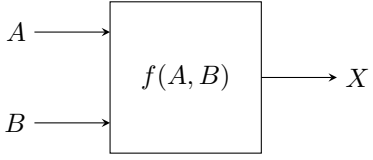
A principal diferença entre a álgebra booleana e a álgebra convencional está no domínio em que elas operam.

- **Álgebra booleana:** Constantes e variáveis só podem assumir dois valores, pertencentes ao conjunto $\mathbb{B} = \{0, 1\}$;
- **Álgebra convencional:** Constantes e variáveis podem assumir infinitos valores, pertencentes a $\mathbb{C}$ e seus subconjuntos.

Valores	
0	1
Falso	Verdadeiro
Não	Sim
Desligado	Ligado
Nível lógico baixo	Nível lógico alto

Uma tabela-verdade é um meio de descrever como a saída de um circuito lógico depende dos níveis lógicos presentes nas entradas desse circuito.

Operador lógico (porta lógica) qualquer de duas entradas e uma saída, $f : \mathbb{B}^2 \to \mathbb{B}$



Simulações

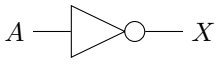
Recomenda-se que todos os circuitos apresentados sejam simulados em logisim.app

PORTA LÓGICA NOT

Operador booleano NOT

A saída é o inverso da entrada:

$$f(A) = \overline{A}.$$

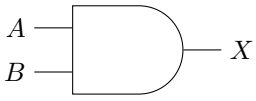


A	X
0	1
1	0

Operador booleano AND

A saída é 1 se todas as entradas forem 1:

$$f(A, B) = A \cdot B.$$



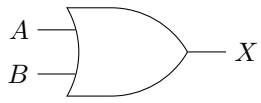
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

PORTA LÓGICA OR

Operador booleano OR

A saída é 1 se pelo menos uma das entradas é 1:

$$f(A, B) = A + B.$$

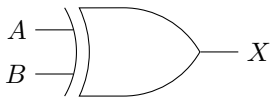


A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Operador booleano XOR

A saída é 1 se as entradas forem diferentes:

$$f(A, B) = A \oplus B.$$



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

NÍVEIS LÓGICOS ATIVOS

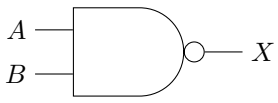
Quando uma linha de entrada ou saída em um circuito lógico não apresenta um inversor (representado por uma bolha), diz-se que essa linha é **ativa em nível alto**.

Quando uma linha de entrada ou saída em um circuito lógico apresenta um inversor (representado por uma bolha), diz-se que essa linha é **ativa em nível baixo**.

Operador booleano NAND

A saída é 0 se todas as entradas forem 1:

$$f(A, B) = \overline{A \cdot B}.$$

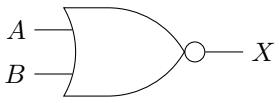


A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

Operador booleano NOR

A saída é 0 se pelo menos uma das entradas é 1:

$$f(A, B) = \overline{A + B}.$$



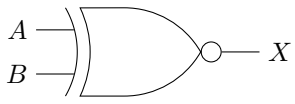
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

PORTA LÓGICA XNOR

Operador booleano XNOR

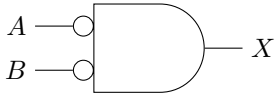
A saída é 1 se as entradas forem iguais:

$$f(A, B) = \overline{A \oplus B}.$$

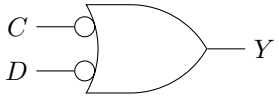


A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

ENTRADAS ATIVAS EM NÍVEL LÓGICO BAIXO

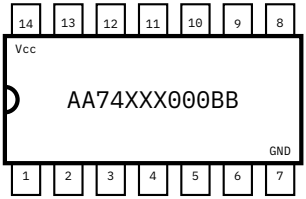


A	B	X
0	0	1
0	1	0
1	0	0
1	1	0



C	D	Y
0	0	1
0	1	1
1	0	1
1	1	0

SÉRIE 7400 DE CIRCUITOS INTEGRADOS



Código	Descrição
AA	Fabricante
74	Classe operacional
XXX	Tipo de tecnologia
000	Função lógica
BB	Tipo de invólucro

Código	Nº de portas/entradas	Porta
7400	4/2	NAND
7402	4/2	NOR
7404	6/1	NOT
7408	4/2	AND
7432	4/2	OR
7486	4/2	XOR

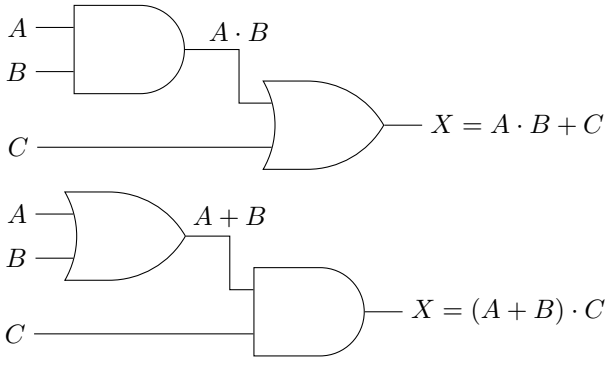
PINAGEM PADRÃO

Nº do Pino	Nome do Pino	Função
1	1A	Entrada A da Porta 1
2	1B	Entrada B da Porta 1
3	1X	Saída da Porta 1
4	2A	Entrada A da Porta 2
5	2B	Entrada B da Porta 2
6	2X	Saída da Porta 2
7	GND	Terra / Neutro
8	3X	Saída da Porta 3
9	3B	Entrada B da Porta 3
10	3A	Entrada A da Porta 3
11	4X	Saída da Porta 4
12	4B	Entrada B da Porta 4
13	4A	Entrada A da Porta 4
14	Vcc	Alimentação Vcc

Caso particular: NOR

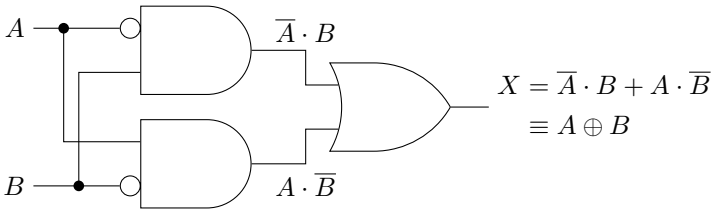
DESCRIÇÃO DE CIRCUITOS LÓGICOS ALGEBRICAMENTE

Qualquer circuito lógico, por mais complexo que seja, pode ser completamente descrito por meio dos 3 operadores booleanos básicos (OR, AND e NOT), que são os elementos fundamentais dos sistemas digitais.



EXEMPLO

O operador XOR pode ser reescrito a partir de operadores básicos.

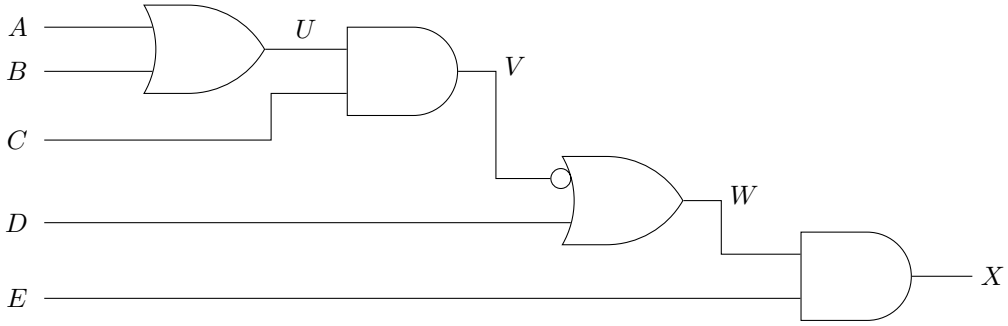


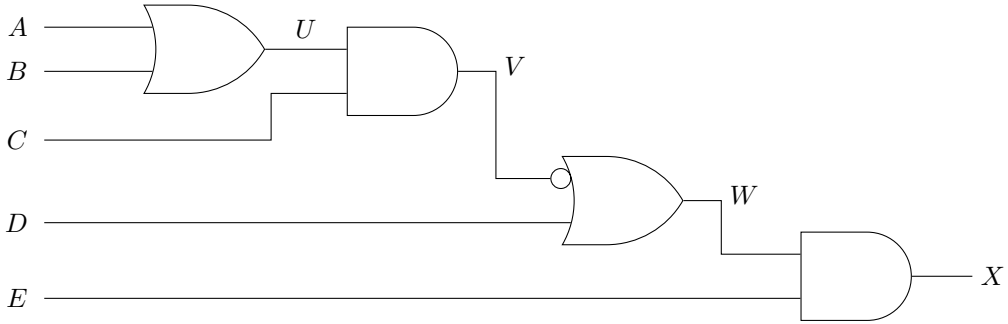
PRECEDÊNCIA DE OPERADORES LÓGICOS

A precedência é apresentada da mais alta para a mais baixa.

Precedência	
NOT	$\overline{A}$
AND	$A \cdot B$
XOR	$A \oplus B$
OR	$A + B$

Parênteses são utilizados para alterar a ordem de precedência padrão, dando prioridade aos operadores contidos neles.


A
B
C
D
E
U
V
W
X



$$U = f(A, B) = A + B$$

$$V = f(U, C) = U \cdot C = (A + B) \cdot C$$

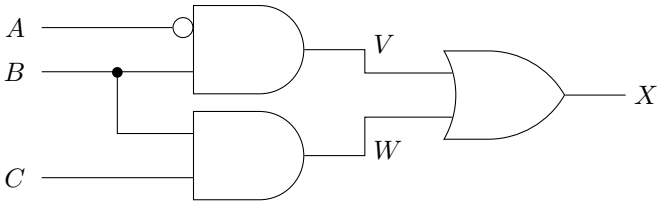
$$W = f(V, D) = \overline{V} + D = \overline{(A + B) \cdot C} + D$$

$$X = f(W, E) = H \cdot E = \left(\overline{(A + B) \cdot C} + D\right) \cdot E$$

ANÁLISE COM TABELA-VERDADE

A tabela-verdade deve mapear todas combinações possíveis de entradas e respectivas saídas de um sistema digital. Para um sistema com n entradas, a tabela conterá $m = 2^n$ possíveis combinações. A cada linha será atribuído um número, de $(0)_2$ a $(m-1)_2$.

Exemplo



Serão $m = 2^3 = 8$ combinações possíveis, contando de 0 a 7.

SOLUÇÃO

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1			
1	0	0	1	1			
2	0	1	0	1			
3	0	1	1	1			
4	1	0	0	0			
5	1	0	1	0			
6	1	1	0	0			
7	1	1	1	0			

SOLUÇÃO

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1			
1	0	0	1	1			
2	0	1	0	1			
3	0	1	1	1			
4	1	0	0	0			
5	1	0	1	0			
6	1	1	0	0			
7	1	1	1	0			

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0		
1	0	0	1	1	0		
2	0	1	0	1	1		
3	0	1	1	1	1		
4	1	0	0	0	0		
5	1	0	1	0	0		
6	1	1	0	0	0		
7	1	1	1	0	0		

SOLUÇÃO

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1			
1	0	0	1	1			
2	0	1	0	1			
3	0	1	1	1			
4	1	0	0	0			
5	1	0	1	0			
6	1	1	0	0			
7	1	1	1	0			

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0		
1	0	0	1	1	0		
2	0	1	0	1	1		
3	0	1	1	1	1		
4	1	0	0	0	0		
5	1	0	1	0	0		
6	1	1	0	0	0		
7	1	1	1	0	0		

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0	0	
1	0	0	1	1	0	0	
2	0	1	0	1	1	0	
3	0	1	1	1	1	1	
4	1	0	0	0	0	0	
5	1	0	1	0	0	0	
6	1	1	0	0	0	0	
7	1	1	1	0	0	1	

SOLUÇÃO

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1			
1	0	0	1	1			
2	0	1	0	1			
3	0	1	1	1			
4	1	0	0	0			
5	1	0	1	0			
6	1	1	0	0			
7	1	1	1	0			

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0		
1	0	0	1	1	0		
2	0	1	0	1	1		
3	0	1	1	1	1		
4	1	0	0	0	0		
5	1	0	1	0	0		
6	1	1	0	0	0		
7	1	1	1	0	0		

nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0	0	
1	0	0	1	1	0	0	
2	0	1	0	1	1	0	
3	0	1	1	1	1	1	
4	1	0	0	0	0	0	
5	1	0	1	0	0	0	
6	1	1	0	0	0	0	
7	1	1	1	0	0	1	

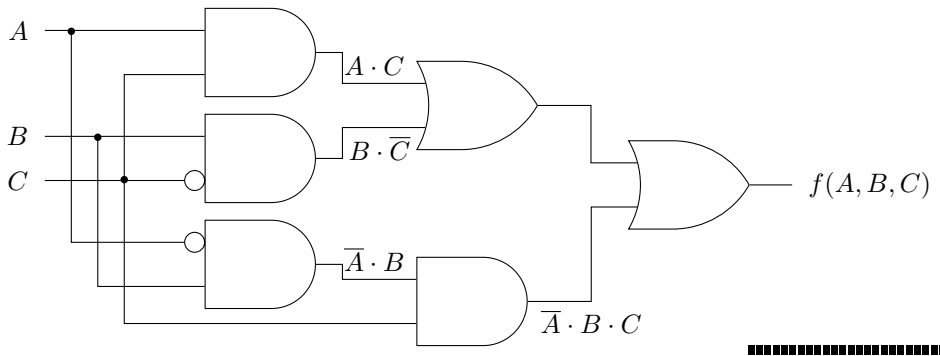
nº	A	B	C	$\overline{A}$	V	W	X
0	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
2	0	1	0	1	1	0	1
3	0	1	1	1	1	1	1
4	1	0	0	0	0	0	0
5	1	0	1	0	0	0	0
6	1	1	0	0	0	0	0
7	1	1	1	0	0	1	1

O comportamento de um sistema digital combinacional é descrito formalmente por uma expressão booleana. A partir da qual é possível implementar o respectivo diagrama de portas lógicas.

$$f(A, B, C) = A \cdot C + B \cdot \overline{C} + \overline{A} \cdot B \cdot C$$

O comportamento de um sistema digital combinacional é descrito formalmente por uma expressão booleana. A partir da qual é possível implementar o respectivo diagrama de portas lógicas.

$$f(A,B,C) = A \cdot C + B \cdot \overline{C} + \overline{A} \cdot B \cdot C$$



TEOREMAS DA ÁLGEBRA BOOLEANA

Os teoremas da álgebra booleana são regras matemáticas usadas para simplificar expressões lógicas e projetar sistemas digitais de forma mais eficiente.

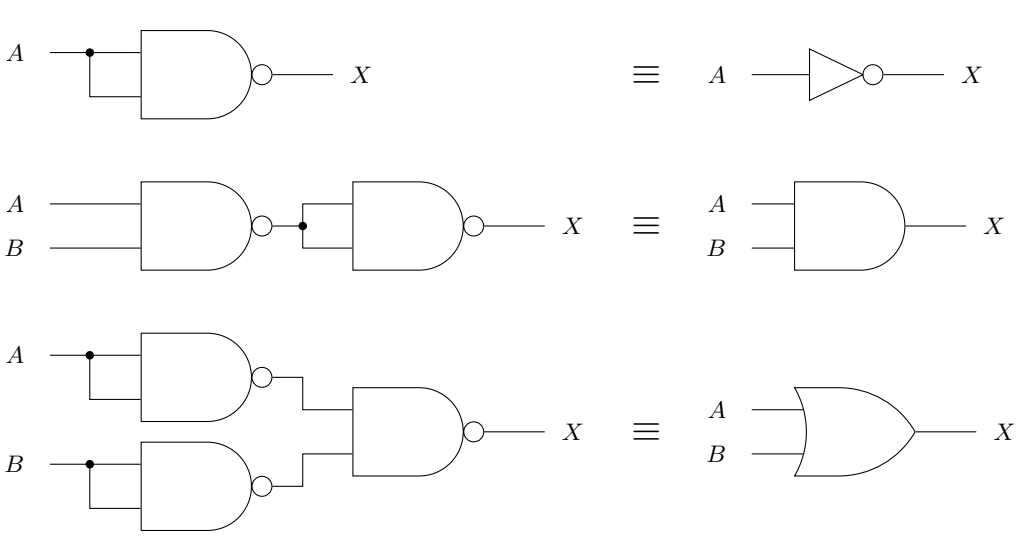
Nome	AND	OR
Involução	$\overline{\overline{A}} = A$	
Elemento neutro	$1 \cdot A = A$	$0 + A = A$
Elemento absorvente	$0 \cdot A = 0$	$1 + A = 1$
Elemento complementar	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Idempotência	$A \cdot A = A$	$A + A = A$
Comutativa	$A \cdot B = B \cdot A$	$A + B = B + A$
Associativa	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributiva	$(A + B) \cdot (C + D) = A \cdot C + A \cdot D + B \cdot C + B \cdot D$	
Aborção	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
Teorema de DeMorgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

UNIVERSALIDADE DAS PORTAS NAND E NOR

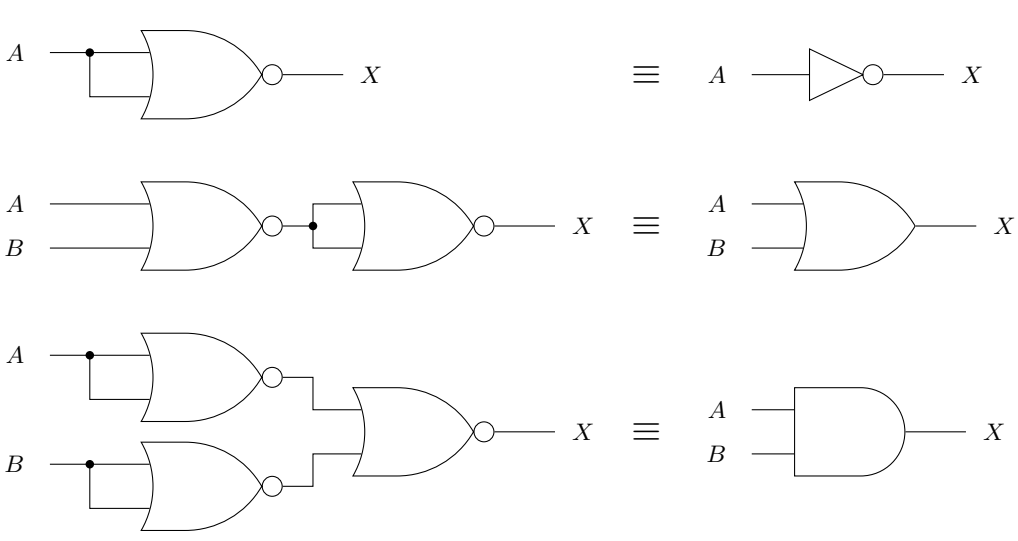
NAND e NOR são portas lógicas funcionalmente completas, ou seja, qualquer expressão lógica pode ser construído usando exclusivamente portas NAND ou NOR.

Operação	NAND	NOR
$\overline{A}$	$\overline{A \cdot A}$	$\overline{A + A}$
$A \cdot B$	$\overline{\overline{A \cdot B}}$	$\overline{\overline{A} + \overline{B}}$
$A + B$	$\overline{\overline{A} \cdot \overline{B}}$	$\overline{\overline{A + B}}$

UNIVERSALIDADE DA PORTA NAND



UNIVERSALIDADE DA PORTA NOR



LISTA DE PROBLEMAS

```
> ls problemas/
> 3.12 (b)
> 3.13
> 3.16 (c), (d), (e), (f), (g), (h)
> 3.28
> 3.29
> 3.32
> 3.33
> 3.34
```